Blastwave: Explosive Force Controller

Physics-driven detonations for Unity. Put a **Detonator** on any object and give it a **Force Profile**. When the fuse runs out, a shock front spreads out from it and pushes, tumbles, breaks loose or vaporizes everything in range. It scales from a frag grenade that knocks a crate stack over up to a scaled-down thermonuclear blast that flattens a warehouse. Blasts leave **scorch marks** on floors and walls and **char** nearby objects. It also ships with sci-fi force fields (singularity, vortex and anti-grav) and an **observer event system** so your own audio, AI, scoring or damage code can react to everything that happens.

Requires Unity 6000.0 or newer. The demo is set up for URP. The runtime code works with any render pipeline (see *Render Pipelines*).

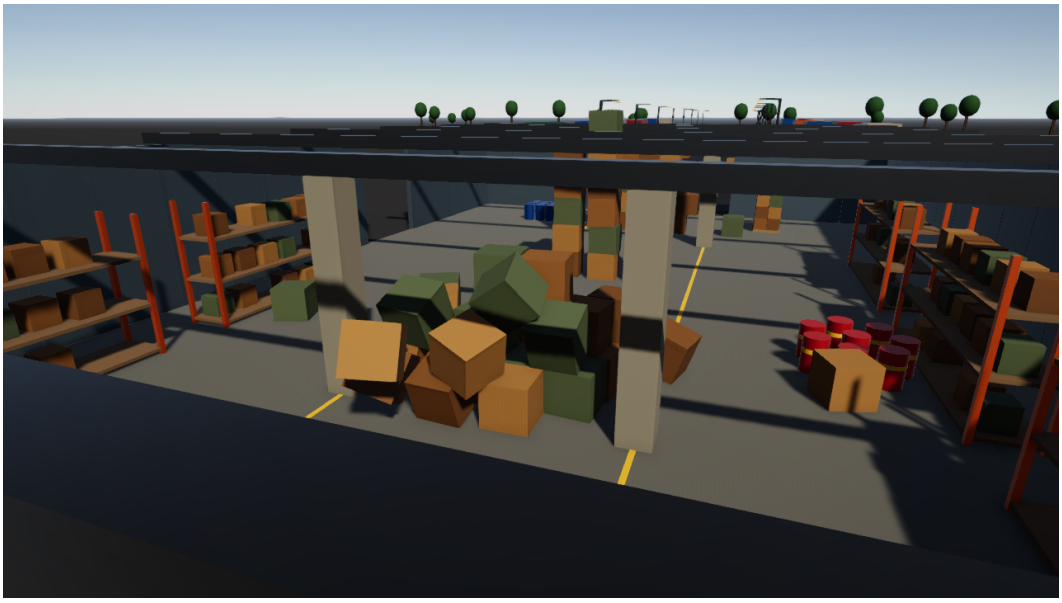
Quick start (60 seconds)

1. **Tools › Blastwave › Setup**. On URP this adds the **Decal** renderer feature that scorch marks need to your URP renderers (it logs each renderer it changes). It also recreates the VFX library and preset profiles if you've deleted them.
2. Select your grenade, barrel or warhead object and choose **Add Component › Blastwave › Detonator**.
3. Assign a profile from `Blastwave/Profiles`, for example *Frag Grenade*, pick a **Trigger** (e.g. *Fuse On Start*) and press Play.

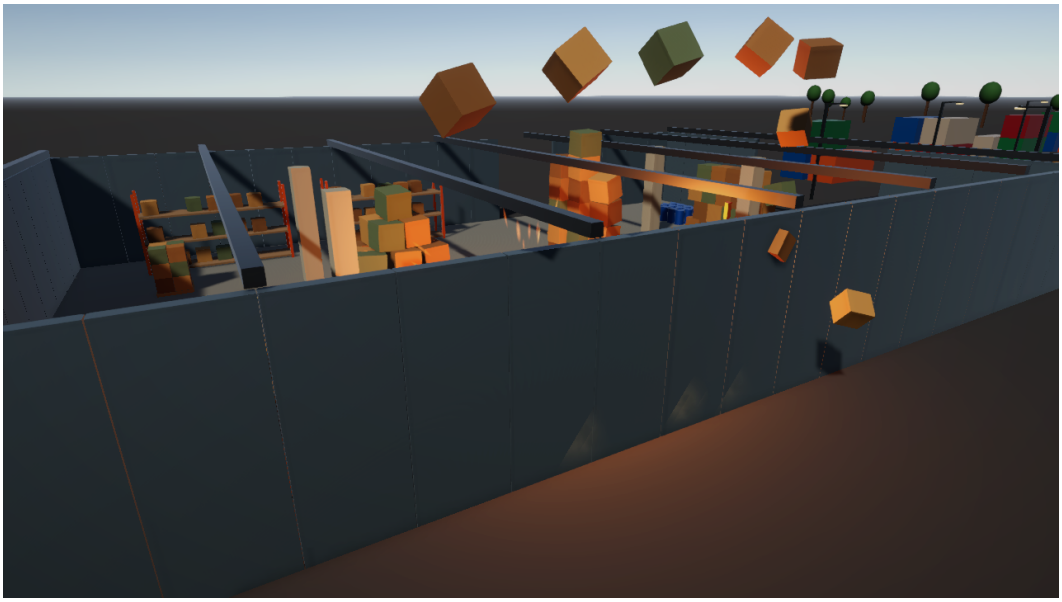
Any object with a Collider and a Rigidbody already reacts. To make a batch of scene objects blastable, select them and use **Tools › Blastwave › Make Selected Objects Blastable**.

Open the demo with **Blastwave/Demo/Scenes/BlastwaveDemo.unity**. To regenerate it, use **Tools › Blastwave › Build Demo Scene**.

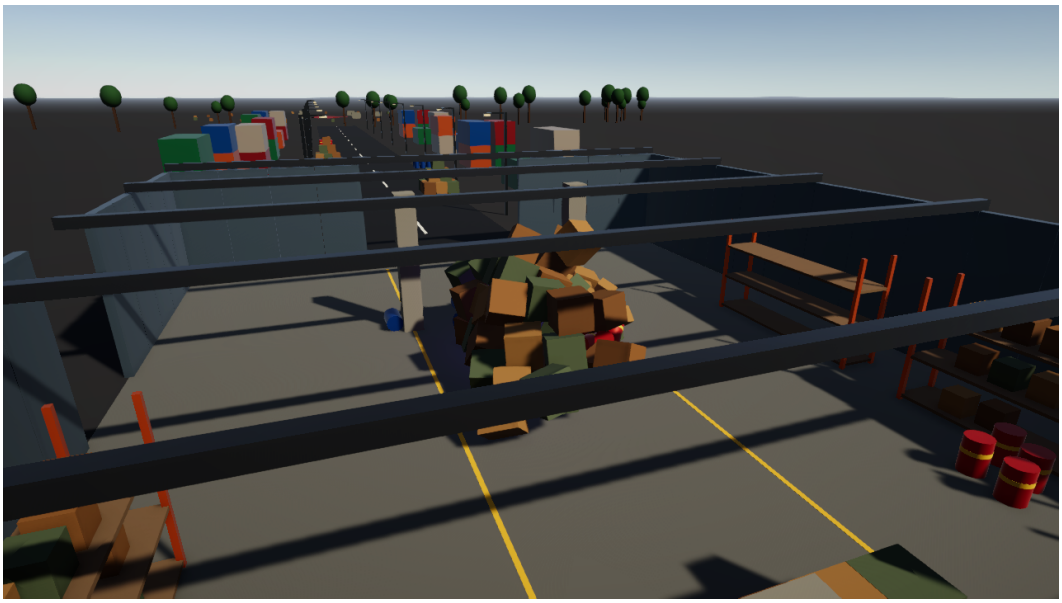
Demo gallery



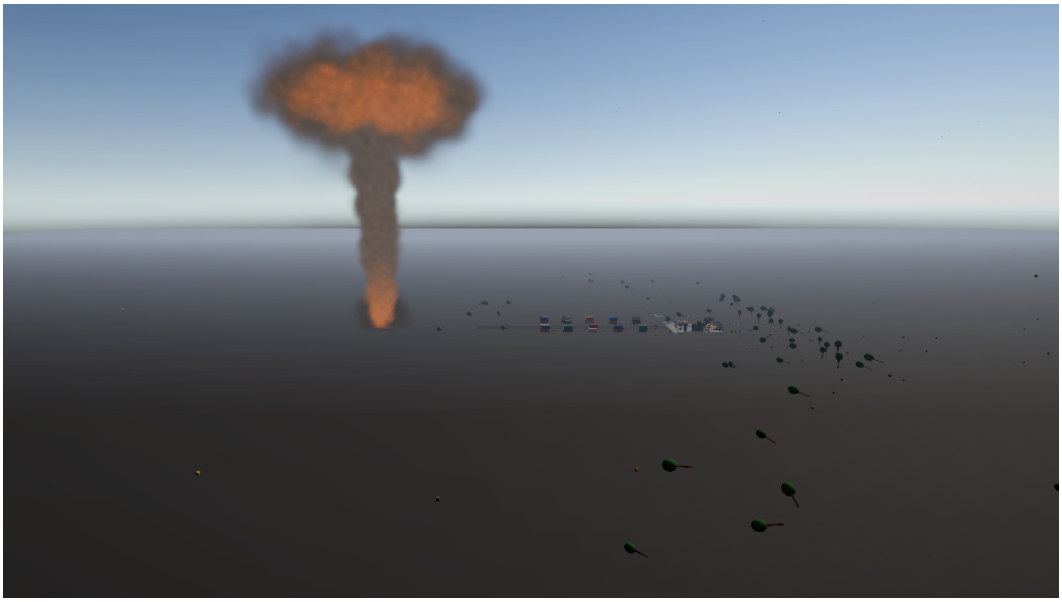
Frag grenade: the crate pyramid is knocked over, not launched.



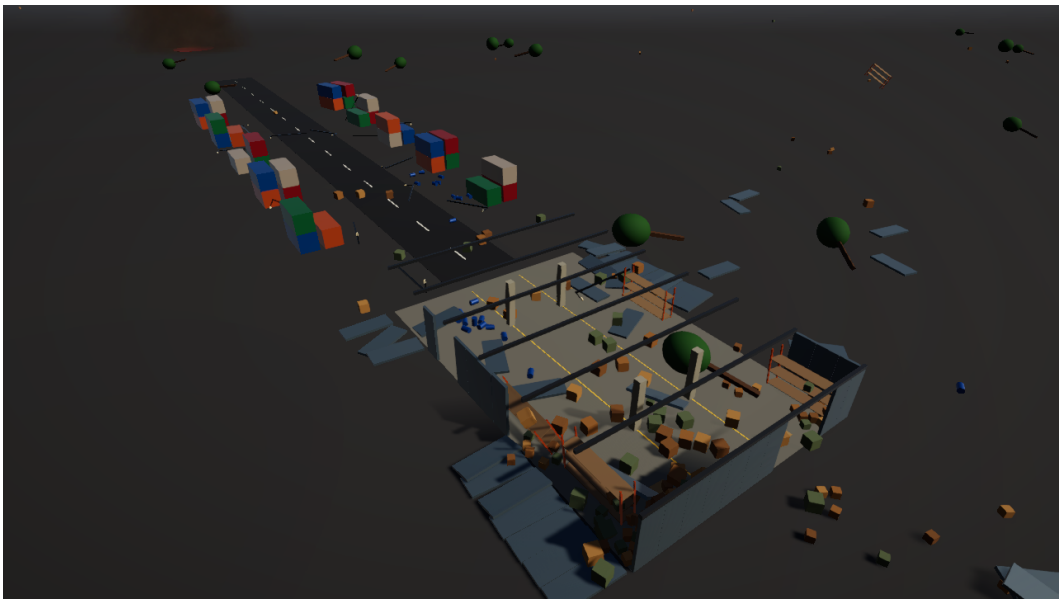
Fuel barrel chain reaction: one barrel sets off the rest.



Singularity grenade: crates and barrels pulled into a single mass.



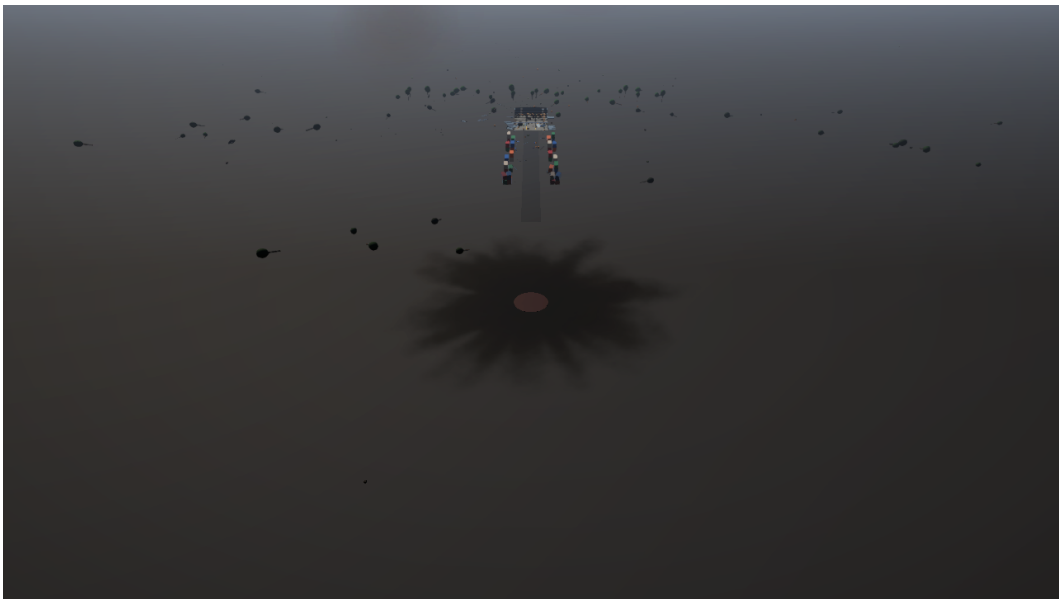
Thermonuclear warhead at 10 s: mushroom cloud, debris and snapped trees.



The warehouse after the warhead: wall panels and racks torn loose.



Scorch decals: a frag grenade's burn mark wrapping from the floor onto the wall.



The warhead's 90 m ground scar around ground zero.

How it works

```
Detonator ──► ForceManager.Detonate(profile, position)
                  │
                  ├─ gathers every collider in radius once, sorted by distance
                  ├─ each FixedUpdate the shock front grows at waveSpeed
                  │      └─ when it reaches a target: falloff → occlusion → vaporize? → IBlastReceivers → impulse
                  ├─ afterwind (negative phase) and updraft act on struck bodies
                  ├─ optional lingering ForceField, follow-up blast, cluster sub-munitions
                  └─ notifies observers (BlastEvents) → camera FX, VFX, scorch decals, charring, your code
```

Every object is struck **exactly once** when the front reaches it. Far objects really are hit later, so a 75 m/s nuclear shock front visibly sweeps across the map.

Force modes

Mode	What `peakPressure` means	Use for
Pressure (default)	Pressure × the object's cross-section facing the blast. Wide, flat objects catch more.	Realistic conventional and nuclear blasts
Impulse	Raw N·s. Heavier objects move less.	Gameplay-tuned knockback
VelocityChange	m/s, ignoring mass	Sci-fi energy weapons, arcade feel

A **negative** `peakPressure` gives an implosion, which pulls objects inward.

`maxVelocityChange` caps how fast any object can be launched, so light props don't end up in orbit.

Breaking things loose

Kinematic bodies with a **ForceReceiver** and a `breakLoad` become dynamic once a blast's **structural load** is at least `breakLoad`. Structural load is |pressure| × the object's largest face area. Blast overpressure loads every face, so a wall that is side-on to a nuke still fails, while a frag 2 m away doesn't. Rough guide for a 2 × 5 m wall panel:

Blast	Structural load
Frag grenade at 2 m	~900
Demolition charge at 5 m	~7,000
Warhead at 220 m	~9,000

Components

Detonator

Field	Notes
Profile	The ForceProfile to detonate.
Trigger	**Fuse On Start**, **Impact** (honors `armDelay` and `impactVelocity`), **Proximity** (radius, layers, optional tag), **Manual**.
Fuse Time	Seconds from `Arm()` to the blast. A burning fuse is never lengthened.
Chain Reaction	Arms itself when another blast hits it with at least `chainThreshold` pressure. This is how fuel barrels set each other off.
Destroy On Detonate	Otherwise the renderers and colliders are hidden.
Fuse Blink	Adds a blinking light that speeds up as the fuse runs out.
Events	`onArmed`, `onDetonated`.

API: `Arm()`, `Arm(float fuse)`, `Disarm()`, `Detonate()`, `IsArmed`, `FuseRemaining`, `HasDetonated`.

ForceReceiver (optional)

Optional per-object tuning: `forceMultiplier`, `vaporizable` / `indestructible`, `breakLoad` + `detachOnBreak`, and **Sway**. Sway gives a static or kinematic mesh, such as a lamp post, sign, tree or antenna, a damped spring tilt around a pivot, driven by blasts and by any ForceField. Events: `onBlast(damage, direction)`, `onBrokeLoose`, `onVaporized`. `TotalDamage` accumulates damage over time.

ForceField

A continuous force volume. Types: **Attract** (gravity well or singularity), **Repel**, **Vortex**, **Directional** (thruster wash or wind tunnel) and **Lift** (anti-grav). Settings: radius, strength (m/s²), falloff curve, duration, fade in/out, `coreRadius` damping, `crushRadius` vaporizing, `maxSpeed`, and an optional glowing visual. A profile can spawn a field when it detonates (`spawnField`). `ForceManager.Instance.SampleFields(position)` returns the combined acceleration at a point, so your own effects can react too.

ForceManager (auto-created)

Steps blasts and keeps a registry of fields. It also cleans up launched debris: bodies that fall below `killHeight`, travel past `killDistance`, or (optionally) stay asleep for `settleCleanupTime` are removed. It grows its query buffer automatically for very large yields.

ForceCameraFX

Put it on your camera to get shake that arrives with the shock front and a white-out flash. The offset is applied only while that camera renders and is removed straight afterwards, so it never fights your camera controller.

ScorchDecals (added to the ForceManager automatically)

Leaves a burn mark on the ground under each blast and, optionally, on walls around it (found with a ring of horizontal raycasts). Only static colliders and kinematic bodies are scorched, so marks never float in mid-air after a crate rolls away.

Field	Notes
Mode	**Auto** uses URP DecalProjectors when the Decal renderer feature is active, otherwise alpha-blended ground quads. Force either with **DecalProjector** or **GroundQuad**.
Max Decals	Budget. When exceeded, the oldest mark is recycled.

Field	Notes
Wall Rays	Rays cast to find walls (0 disables wall scorch for all profiles).
Surface Layers	Which layers can be scorched.
Material overrides	Use your own decal or quad material.

API: `Place(point, normal, radius, opacity, settings)` to add a mark yourself (for example from a weapon impact), `Clear()`, `LiveCount`. In the Editor, if the URP renderer has no Decal feature, a one-time warning tells you to run Setup and marks fall back to quads.

BlastCharring (added to the ForceManager automatically)

Darkens objects close to a blast toward the profile's `charColor`. Char builds up over several blasts to a cap of `maxChar`, and objects behind cover get less (`occludedMultiplier`). It uses MaterialPropertyBlocks, so **shared materials are never modified**. Each charred object gets a `CharState` component; call `CharState.Clear()` to restore it. Set `ForceReceiver.charrable = false` to protect an object.

To turn either effect off globally, remove the component from the ForceManager or untick `ForceManager.addSurfaceEffects`.

Force Profile reference

Group	Fields
Blast	radius, peakPressure, forceMode, falloff curve, upwardsModifier, hitPointTorque, randomSpin, maxVelocityChange, affectedLayers, maxDamage
Shockwave	waveSpeed (0 = instant)
Cover	useOcclusion, occludedMultiplier, occluderLayers
Fireball core	vaporizeRadius
Afterwind	strength, delay, duration (air rushing back toward the center after the front passes)
Updraft	strength, radius, duration, swirl (the fireball column that lifts debris)
Lingering field	spawnField + ForceFieldSettings
Follow-up / cluster	followUp + delay, clusterProfile + count + spread + delay range
Visuals	flash, fireball, sparks, smoke, dust ring, shock shell, mushroom cloud
Surface damage	scorch on/off, scorchRadius, scorchOpacity, wallScorch, scorchLifetime (0 = permanent), scorchFadeTime, scorchTint (see note), charRadius (fraction of blast radius), charAmount, charColor
Camera	shake strength, range, duration, screen flash

Note: URP's built-in decal shader has no tint input, so `scorchTint` only shows on ground-quad scorch marks, or on your own decal material if it has a tint property.

The profile inspector also shows a reference table: for several distances, the pressure, the Δv a 1 m / 20 kg crate gets and when the shock front arrives.

Bundled presets

Frag Grenade · Concussion Charge · Fuel Barrel · Demolition Charge · Plasma Charge · Singularity Grenade (+ Collapse) · Vortex Grenade · Anti-Grav Pulse · Cluster Bomb (+ Bomblet) · Thermonuclear Warhead.

The warhead is **scaled down to fit a game scene**: 320 m reach, 30 m vaporize core, 75 m/s shock front. Raise `radius` and `waveSpeed` for realistic scale.

Scripting

```
using Blastwave;

// Fire any profile anywhere
ForceManager.Detonate(myProfile, hitPoint, instigator: gameObject);

// Per-object hook: implement on the struck object itself
public class Health : MonoBehaviour, IBlastReceiver
{
    public float hp = 100;
    public void OnBlast(in BlastHit hit)
    {
        hp -= hit.damage;            // falloff- and cover-scaled
        if (hit.vaporized) Die();
    }
}
```

`BlastHit` contains: blastId, origin, point, direction, distance, normalizedDistance, pressure, impulse, velocityChange, structuralLoad, damage, occluded, vaporized, rigidbody, collider.

Events and hooks (observer pattern)

`IBlastReceiver` is for the object being hit. To react to blasts **anywhere**, from an audio manager, AI director, score system, decal spawner or network layer, use the observer system:

- **Subject:** `BlastEvents`, a global registry. `Subscribe(observer, order)`, `Unsubscribe(observer)`, `IsSubscribed`, `ObserverCount`.
- **Observer:** `IBlastObserver`. Every callback has an empty default, so implement only the ones you need.
- **Convenience base:** `BlastObserverBehaviour` subscribes in `OnEnable` and unsubscribes in `OnDisable`.
- **No-code option:** the **Blast Event Relay** component exposes the same events as UnityEvents, filtered by profile, distance, or "only this object".

Callback	When
`OnDetonated(in BlastInfo)`	The moment a blast goes off, before its shock front travels.
`OnBodyStruck(in BlastHit)`	The shock front hit an object (after force was applied). Once per object per blast.
`OnVaporized(in BlastHit)`	An object was inside a fireball core.
`OnBrokeLoose(ForceReceiver, in BlastHit)`	A pinned object exceeded its break load and is now free.
`OnBlastFinished(in BlastInfo)`	The shock front and the afterwind and updraft phases are complete.
`OnFieldStarted(ForceField)` / `OnFieldEnded(ForceField)`	A force field became active or ended.

```
using Blastwave;

// A MonoBehaviour observer: auto-subscribes while enabled.
public class ExplosionAudio : BlastObserverBehaviour
{
    public AudioClip boom, debrisImpact;

    public override void OnDetonated(in BlastInfo info)
    {
        // Delay the sound by distance so it arrives with the shock front.
        float d = Vector3.Distance(Camera.main.transform.position, info.position);
        StartCoroutine(PlayAfter(boom, info.position, info.profile.ArrivalTime(d)));
    }

    public override void OnBrokeLoose(ForceReceiver receiver, in BlastHit hit) =>
        AudioSource.PlayClipAtPoint(debrisImpact, receiver.transform.position);
}

// A plain C# observer (no GameObject needed), e.g. a score system.
public class BlastScore : IBlastObserver
{
    public int score;
    public BlastScore() => BlastEvents.Subscribe(this);
    public void Dispose() => BlastEvents.Unsubscribe(this);
    public void OnVaporized(in BlastHit hit) => score += 100;
}
```

Guarantees

- **Ordering:** observers are notified in ascending `order`; ties go in subscription order. The built-in scorch and charring observers use order 0.
- **Safe changes during a callback:** observers can subscribe or unsubscribe inside a callback. The change takes effect from the next notification.
- **Fault isolation:** if an observer throws, the error is logged with the observer's type, and the blast and every other observer carry on.
- **Correlation:** every `BlastHit.blastId` matches the `BlastInfo.id` of the detonation that caused it, so you can total hits or damage per blast (see `Demo/Scripts/BlastEventFeed.cs`).
- **Destroyed observers:** destroyed MonoBehaviour observers are skipped automatically.

Your own VFX or audio: turn off `ForceManager.spawnVisuals`, or turn off `visuals.enabled` on a profile, and implement `OnDetonated`. The built-in scorch and charring are themselves observers, so you can replace them the same way. The simple `ForceManager.Detonated` C# event is still available for one-liners.

Render pipelines

- **URP**: fully supported. The demo uses URP Lit materials and a Bloom/ACES post-processing volume. Scorch decals use URP's Decal renderer feature, which Setup adds.
- **Built-in / HDRP**: the force system and camera FX don't depend on the render pipeline. The procedural VFX look for a suitable particle shader (`Particles/Standard Unlit`, `HDRP/Unlit`). For player builds, make sure the materials in `Runtime/VFX/Assets` use a shader that your pipeline includes. The demo materials need converting with your pipeline's material upgrader. Scorch marks automatically use the ground-quad fallback. Charring works on any material with a `_BaseColor` or `_Color` property.

Performance notes

- One overlap query per detonation, not per frame. Struck targets are sorted once.

- Occlusion costs one raycast per struck target. Turn off `useOcclusion` on profiles that are spammed a lot.
- Large yields touch every body in range. The ~430-body demo runs the warhead comfortably on desktop.
- ForceFields run an overlap query every FixedUpdate. Keep their radii sensible.
- Scorch costs one downward raycast plus `wallRays` raycasts per detonation, and is capped by `maxDecals`. Charring touches only objects inside `charRadius`.
- Charring uses MaterialPropertyBlocks, which takes those renderers out of the SRP Batcher. That's fine for props; set `charrable = false` on large numbers of identical objects if draw calls matter.

Demo controls

`1`–`0` pick explosive · **LMB** throw/place · **RMB** orbit · **WASD/QE** move · **Wheel** zoom · **C** camera bookmarks · **Space** detonate all armed · **Z** slow motion · **R** reset · **H** hide help.

The event feed in the bottom-right corner is `BlastEventFeed`, an example observer. Explosives spawned by the demo are placeholder primitives. Assign your own models (from Blender, for example) to the `prefab` slot of each entry on *Blastwave Demo Controller*.

Tests

`Blastwave/Tests` has 24 PlayMode tests. They cover the physics (push, radius, wave delay, occlusion, chain reaction, fuse, vaporize, indestructible, break-loose, fields, implosion, area-based pressure). They also cover the observer system (delivery, unsubscribe, duplicate subscribe, ordering, fault isolation, every event type, relay filters) and surface damage (ground and wall scorch, decal budget, fade-out, charring radius, opt-out, accumulation cap, shared materials untouched). Run them from **Window › General › Test Runner › PlayMode**.

Troubleshooting

Missing-file errors in `Library/PackageCache` after import

Symptoms: `DirectoryNotFoundException`, `Could not find a part of the path`, or `error code 206` in the Console. These come from Unity's own packages, not Blastwave. Windows limits file paths to 260 characters, and some package files sit deep inside `Library/PackageCache`, so a project in a deeply nested folder can push them over the limit. Either fix works:

- Move the project to a shorter path, such as `D:\Projects\MyGame`, then delete its `Library` folder and reopen it.
- Or turn on Windows long-path support. Set `HKLM\SYSTEM\CurrentControlSet\Control\FileSystem\LongPathsEnabled` to `1`, or enable the Group Policy *Computer Configuration › Administrative Templates › System › Filesystem › Enable Win32 long paths*. Then restart Unity.

Scorch marks look like flat discs

You may also see the warning "URP renderer has no active Decal feature". Run **Tools › Blastwave › Setup**. It adds URP's Decal renderer feature to your URP renderers, and after that scorch marks use real decals. Until then they fall back to ground quads, which work but don't wrap onto uneven surfaces. If you switch to a renderer created later, run Setup again. It's safe to run any number of times.